CS341 Mini Project: Tiered Array Problem Set

Data Structures and Algorithms, Choose Your Own Difficulty

How This Works

All problems in this project use arrays (1D arrays or 2D grids), with no linked lists this round. The problems are grouped into three topics, and each topic offers a problem at three difficulty levels: Easy (20 points), Medium (40 points), and Hard (60 points).

You choose exactly **three** problems in total, from anywhere in the pool of nine. You are not required to touch all three topics, and you may take more than one problem from the same topic.

- Your three problems must add up to **at least 100 points**, and the total may **not exceed 120 points**.
- You may select at most two Easy problems, at most two Hard problems, and at most three Medium problems across your three picks.

For each of your three chosen problems, submit both a **brute force** solution and the **most optimized** solution you can justify, each with a written time and space complexity analysis, following the same two part format as the base project proposal.

Constraints carried over from the base proposal still apply: built in sort functions and structures based on hashing are disallowed in the main solution (optional stretch tier only), and the implementation language matches whatever has been used in labs to date.

The Problem Pool at a Glance

Topic	Easy (20)	Medium (40)	Hard (60)
1. Holding Water	1A Tea Urn Rush	1B Two Floodwalls	1C Rooftop Puddles
2. Grid Territories	2A Fencing the Pasture	2B Forest Patches	2C Reclaim One Plot
3. Rolling Windows	3A Warmest Stretch	3B Food Truck's Best Run	3C Rolling Peak Monitor

Topic 1: Holding Water

1A Tea Urn Rush

Easy · 20 points

A campus tea stall has one urn with two spouts, and it serves three drinks: mint tea, coffee, and hot chocolate. In one minute the urn can either pour a single cup of any drink, or pour two cups at the same time as long as the two cups are *different* drinks (the two spouts cannot run the same drink together).

A queue of orders arrives before class. Given how many cups of each drink are needed, find the fewest minutes the stall needs to pour every cup.

Input. An array `orders` of length 3, where `orders[0]`, `orders[1]`, `orders[2]` are the numbers of mint tea, coffee, and hot chocolate cups ordered.

Output. The minimum number of minutes needed to pour all the cups.

Example.

`orders = [3, 5, 1]` ⇒ **5 minutes**.

Coffee is the bottleneck: only one coffee can be poured per minute, so at least 5 minutes are unavoidable. The schedule below reaches that bound (M = mint tea, C = coffee, H = hot chocolate).

	min 1	min 2	min 3	min 4	min 5
Spout 1	M	M	M	H	–
Spout 2	C	C	C	C	C

1B Two Floodwalls

Medium · 40 points

A town is building a temporary holding pond along a riverbank. Engineers have already set up a straight row of thin vertical floodwalls, evenly spaced one meter apart, but the walls have different heights. The plan is to pick exactly two of the walls and let water fill the space between them. The water level can rise only as high as the *shorter* of the two chosen walls, and the walls standing in between do not block anything (they are thin enough to ignore).

Pick the pair of walls that holds the most water, measured as a two-dimensional cross section: height of the water times the distance between the two walls.

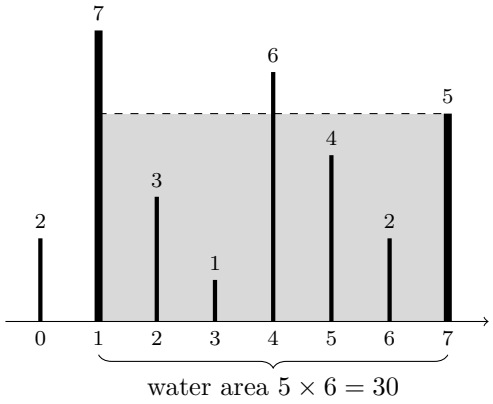
Input. An array `walls` of $n \geq 2$ non-negative integers, where `walls[i]` is the height of the wall at position i.

Output. The largest cross-sectional area of water any pair of walls can hold, that is, the maximum over $i < j$ of $\min(\texttt{walls[i]}, \texttt{walls[j]}) \times (j - i)$.

Example.

`walls = [2, 7, 3, 1, 6, 4, 2, 5]` ⇒ **30**.

Choosing the walls at positions 1 and 7 (heights 7 and 5) gives a water level of 5 across a distance of 6, for an area of $5 \times 6 = 30$. No other pair does better; for example, positions 1 and 4 give only $6 \times 3 = 18$.



1C Rooftop Puddles

Hard · 60 points

A street is lined with flat-roofed buildings standing side by side, each one exactly one unit wide. After a heavy storm, rainwater collects in the dips between taller buildings and sits there. Water above a building stays only if there is a building at least as tall somewhere to its left *and* somewhere to its right to hold it in; anything else runs off the ends of the street.

Given the building heights from left to right, compute how many units of water remain sitting on the street after the storm.

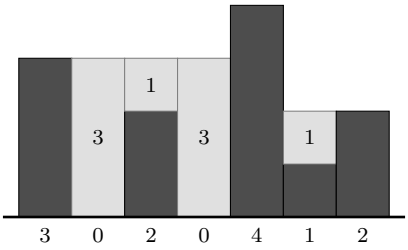
Input. An array `heights` of n non-negative integers, one per building, each building of width 1.

Output. The total units of water held between the buildings.

Example.

`heights = [3, 0, 2, 0, 4, 1, 2]` ⇒ **8 units**.

The water above each position is $[0, 3, 1, 3, 0, 1, 0]$. For instance, position 3 has height 0, the tallest building to its left is 3 and the tallest to its right is 4, so the water there rises to $\min(3, 4) = 3$ units. The shaded cells in the figure are water; the dark bars are buildings.



Topic 2: Grid Territories

All three problems in this topic use a 2D grid of 0s and 1s. Two cells count as *connected* only if they share a side (up, down, left, or right); touching at a corner does not count. In the figures, dark cells are 1s and white cells are 0s. Rows and columns are numbered from 0.

2A Fencing the Pasture

Easy · 20 points

A farmer's land is surveyed as a grid of 1×1 plots, where `1` marks pasture and `0` marks bare ground. All of the pasture forms a single connected region, and it has no enclosed pockets of bare ground inside it. The farmer wants to run a fence along the entire outer border of the pasture so the herd cannot wander off.

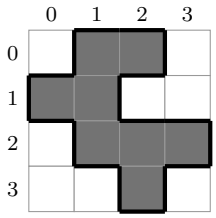
How many units of fencing does the farmer need?

Input. A 2D grid `land` of 0s and 1s containing exactly one connected region of 1s with no holes.

Output. The length of the region's outer boundary, where each exposed side of a pasture plot contributes 1 unit.

Example.

```
land = [[0,1,1,0],
        [1,1,0,0],
        [0,1,1,1],
        [0,0,1,0]]
```



$\Rightarrow$ **18 units of fence**.

The pasture has 8 plots. The thick line in the figure is the fence; counting its unit segments gives 18. Equivalently, each plot contributes one unit for every side that faces bare ground or the edge of the map.

2B Forest Patches

Medium · 40 points

A satellite photo of a region has been turned into a grid, where `1` marks a cell covered by trees and `0` marks open land. A conservation team wants to know how many separate patches of forest there are, because each patch will get its own ranger.

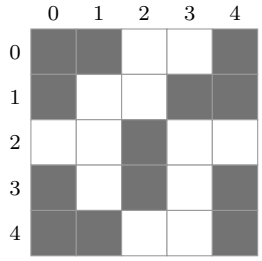
A patch is a maximal group of connected tree cells. Count the patches.

Input. A 2D grid `map` of 0s and 1s, of size $r \times c$.

Output. The number of distinct forest patches.

Example.

```
map = [[1,1,0,0,1],
       [1,0,0,1,1],
       [0,0,1,0,0],
       [1,0,1,0,1],
       [1,1,0,0,1]]
```



$\Rightarrow$ **5 patches**.

The patches are: the top-left group of 3 cells, the top-right group of 3, the vertical pair in the middle column, the bottom-left group of 3, and the pair on the right edge of the bottom two rows. Note that the cell at row 1, column 3 and the cell at row 2, column 2 only touch at a corner, so they are in different patches.

2C Reclaim One Plot

Hard · 60 points

A coastal city is an $n \times n$ grid where `1` is land and `0` is water. The city has budget for exactly one land reclamation project: it may fill in *at most one* water cell, turning it into land. The goal is to create the largest single connected landmass possible, since that is where the new stadium will go.

Return the size (number of cells) of the largest connected landmass the city can have after at most one reclamation.

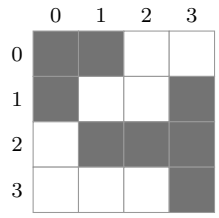
Input. An $n \times n$ grid `city` of 0s and 1s.

Output. The largest possible size of a connected region of 1s after changing at most one 0 into a 1. If the grid has no water, this is simply n^2.

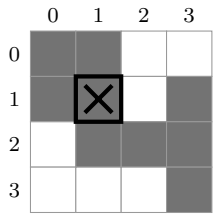
Example.

`city = [[1,1,0,0],[1,0,0,1],[0,1,1,1],[0,0,0,1]]` $\Rightarrow$ **9**.

Before any change there are two landmasses, one of size 3 (top left) and one of size 5 (right and bottom). Filling the water cell at row 1, column 1 (marked with an X) touches both of them and merges them into a single landmass of $3 + 5 + 1 = 9$ cells. (Filling row 2, column 0 would also reach 9; any correct answer returns the size, not the cell.)



Before: sizes 3 and 5



After filling (1,1): size 9

Topic 3: Rolling Windows

All three problems in this topic look at contiguous runs of an array, that is, consecutive positions with nothing skipped. In the figures, the shaded, outlined cells are the run being described. Positions are numbered from 0.

3A Warmest Stretch

Easy · 20 points

A weather station logs one temperature reading (in °C) per day. A travel agency wants to advertise "the warmest k days in a row" from last season, so it needs the block of exactly k consecutive days with the highest average temperature.

Return that highest average.

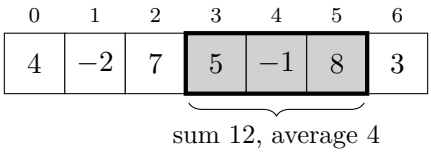
Input. An array `temps` of n integers and an integer k with $1 \le k \le n$.

Output. The maximum average over all blocks of exactly k consecutive readings (a real number).

Example.

`temps = [4, -2, 7, 5, -1, 8, 3]`, `k = 3` $\Rightarrow$ **4.0**.

The five blocks of 3 days have sums $9, 10, 11, 12, 10$. The best is positions 3 to 5, with sum $5 + (-1) + 8 = 12$ and average $12/3 = 4.0$.



3B Food Truck's Best Run

Medium · 40 points

A food truck owner records the net profit for each day of the semester. Some days make money and some days lose it (rain, closed roads, spoiled stock). For an end-of-year write-up, the owner wants to find the streak of consecutive days that earned the most money in total. The streak must contain at least one day, and it may be any length.

Return the total profit of the best streak.

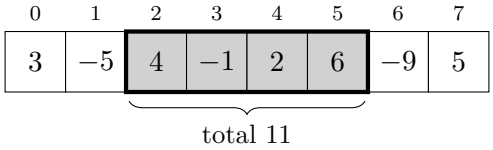
Input. An array `profit` of $n \ge 1$ integers, which may be positive, negative, or zero.

Output. The largest possible sum of a non-empty run of consecutive entries.

Example.

`profit = [3, -5, 4, -1, 2, 6, -9, 5]` ⇒ **11**.

The best streak is positions 2 to 5: $4 + (-1) + 2 + 6 = 11$. Extending it to the left picks up the -5 loss, and extending it to the right picks up the -9 loss, so neither helps. If every day is a loss, the answer is the single least bad day.



3C Rolling Peak Monitor

Hard · 60 points

A noise sensor near a construction site produces one reading per minute. The site's dashboard shows a "recent peak": the loudest reading among the last k minutes. Once the first k readings are in, the dashboard updates every minute as the window moves forward by one position, dropping the oldest reading and adding the newest.

Produce the full list of values the dashboard displays.

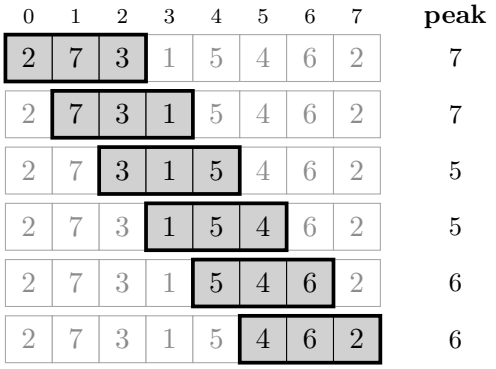
Input. An array `readings` of n integers and a window size k with $1 \le k \le n$.

Output. An array of length $n - k + 1$ whose entry i is the maximum of `readings[i]` through `readings[i+k-1]`.

Example.

`readings = [2, 7, 3, 1, 5, 4, 6, 2]`, `k = 3` ⇒ **[7, 7, 5, 5, 6, 6]**.

Each row of the figure is one position of the window, from the first three readings down to the last three, with the peak the dashboard shows on the right. Notice that the 7 stays on the dashboard for two updates and then drops out of the window, after which the peak falls to 5.



Valid Point Combinations

With the rules above (exactly 3 problems, a total between 100 and 120 points inclusive, at most two Easy, at most two Hard, at most three Medium), these are the only combinations that qualify:

Combination	**Breakdown**	**Total**	**Notes**
2 Easy + 1 Hard	$20 + 20 + 60$	100	Meets the minimum exactly
1 Easy + 2 Medium	$20 + 40 + 40$	100	Meets the minimum exactly
1 Easy + 1 Medium + 1 Hard	$20 + 40 + 60$	120	One of each, reaches the cap
3 Medium	$40 + 40 + 40$	120	All Medium, reaches the cap